



Instituto Tecnológico de Mazatlán



**Control de acceso con lector RFID RC522
por bus SPI con Arduino UNO R4 WiFi**

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Barrón Hernández

Integrantes:

Ximena Armenta Dávila

Carlos Tadeo Ibarra Castañeda

Dulce Princesa Ulivarria González

1. Introducción

La identificación por radiofrecuencia (RFID) permite reconocer objetos sin contacto físico: una etiqueta pasiva, como una tarjeta o un llavero, se energiza con el campo magnético del lector y le responde con un número de identificación único (UID). El módulo RC522 está basado en el circuito MFRC522 de NXP, trabaja a 13.56 MHz y lee etiquetas del tipo MIFARE. El chip puede comunicarse por SPI, I2C o UART, pero el módulo que se utilizó viene configurado para SPI.

El bus SPI (Serial Peripheral Interface) es un protocolo síncrono de tipo maestro-esclavo. El maestro, en este caso el Arduino, genera el reloj (SCK), envía datos por MOSI, recibe datos por MISO y elige con qué dispositivo habla por medio de una línea de selección (CS o SS). A diferencia de I2C, los dispositivos no tienen dirección: cada uno se selecciona con su propio pin CS, y la transmisión puede ir en ambos sentidos al mismo tiempo (full-duplex).

En esta práctica se conectó el lector RC522 a un Arduino UNO R4 WiFi por el bus SPI, se leyó el UID de dos llaveros RFID, se mostró en el Monitor Serie en formato hexadecimal y se usó para simular un control de acceso: si el UID coincide con el autorizado se enciende un LED verde durante 2 segundos, controlados con `millis()`, y si no coincide se muestra “ACCESO DENEGADO”.

2. Objetivos

Objetivo general:

- Comunicar el Arduino con un lector RFID RC522 por medio del bus SPI, leer el UID de una etiqueta, mostrarlo en el Monitor Serie y usarlo para simular un control de acceso.

Objetivos específicos:

- Instalar la librería MFRC522 desde el Library Manager e inicializar el bus SPI y el lector.
- Confirmar al arrancar, en el Monitor Serie, si existe comunicación con el lector leyendo su registro de versión.
- Mostrar el UID de cada etiqueta en hexadecimal, con cada byte separado por un espacio (por ejemplo, UID : 99 EB 7B 63).
- Guardar en el programa el UID autorizado y compararlo con el de cada etiqueta leída.
- Encender el LED de acceso durante 2000 ms usando `millis()`, sin detener la lectura de etiquetas.

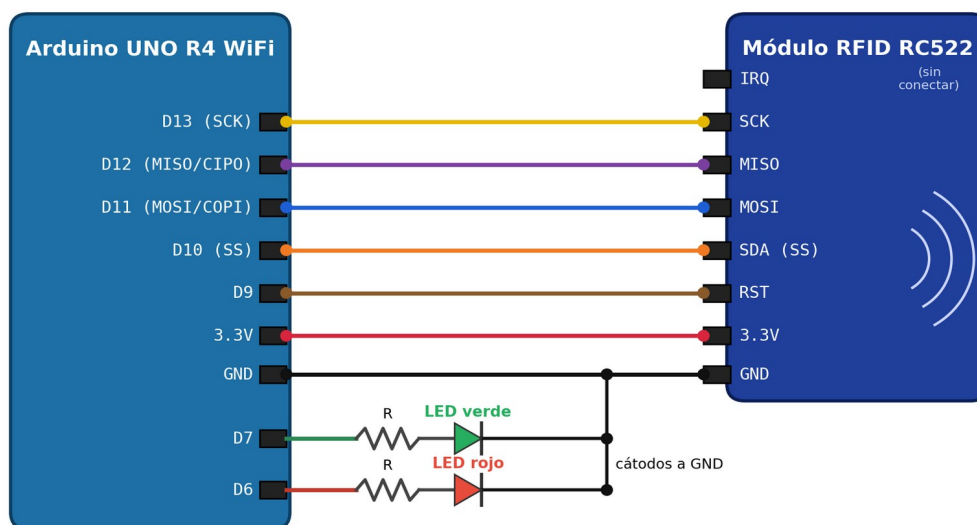
3. Herramientas y material utilizado

- 1 Arduino UNO R4 WiFi y su cable USB
- 1 módulo lector RFID RC522 (alimentado a 3.3 V)
- 2 llaveros RFID de 13.56 MHz
- 1 LED verde (acceso permitido) y 1 LED rojo (acceso denegado), cada uno con su resistencia limitadora
- 1 protoboard y cables de conexión (jumpers)
- Arduino IDE 2 con las librerías SPI (incluida) y MFRC522 (GithubCommunity), y Monitor Serie a 9600 baudios

4. Desarrollo / Metodología

Primero se instaló la librería **MFRC522** desde el Library Manager del Arduino IDE. Después se conectó el lector al Arduino usando los pines SPI del UNO R4 WiFi (D13, D12 y D11), el pin D10 como selección del lector y el

D9 para su reinicio. El módulo se alimentó desde el pin de **3.3 V** del Arduino, ya que conectarlo a 5 V puede dañarlo. Los LEDs se conectaron con sus resistencias a los pines D7 (verde) y D6 (rojo), con los cátodos a GND.



SPI: SCK, MOSI y MISO son líneas compartidas del bus; SS (SDA) selecciona al lector. El RC522 se alimenta con 3.3 V.

Figura 1. Diagrama de conexiones entre el Arduino UNO R4 WiFi, el módulo RC522 y los LEDs indicadores.

La Tabla I resume las conexiones y la función de cada línea.

Tabla I. Conexiones del módulo RC522 y de los LEDs con el Arduino UNO R4 WiFi.

Pin del módulo	Pin del Arduino	Función
SDA (SS)	D10	Selección del lector (Chip Select). Activo en bajo.
SCK	D13	Reloj del bus SPI generado por el Arduino.
MOSI	D11	Datos del Arduino hacia el lector (comandos y registros).
MISO	D12	Datos del lector hacia el Arduino (versión, UID, estado).
IRQ	—	Interrupción del lector. No se utilizó.
GND	GND	Tierra común.
RST	D9	Reinicio del lector por hardware.
3.3V	3.3V	Alimentación del módulo (no conectar a 5 V).
LED verde	D7	Se enciende 2 s cuando el acceso es permitido.
LED rojo	D6	Se enciende 2 s cuando el acceso es denegado.

El programa (`control_acceso_rfid.ino`) funciona de la siguiente manera:

- **Arranque:** en `setup()` se abre el Monitor Serie, se inicia el bus con `SPI.begin()` y se configura el lector con `iniciarLector()`, que da un pulso de reinicio en RST, ejecuta `PCD_Init()` y pone la ganancia de la antena al máximo. Luego se lee el registro `VersionReg`: si devuelve `0x00` o `0xFF` no hay comunicación y se muestra “ERROR: No hay comunicacion con el lector RC522”; en caso contrario se muestra “Comunicacion OK con el lector RC522”.
- **Lectura:** en cada vuelta de `loop()` se llama a `PICC_IsNewCardPresent()` y `PICC_ReadCardSerial()`; ninguna de las dos bloquea el programa. Cuando hay una etiqueta,

mostrarUID() imprime cada byte en hexadecimal separado por un espacio y agrega un 0 a la izquierda cuando el byte es menor a 0x10.

- **Comparación:** uidCoincide() compara el tamaño y cada byte del UID leído con el arreglo uidAutorizado[] = {0x99, 0xEB, 0x7B, 0x63}. Si coinciden se imprime “ACCESO PERMITIDO” y se enciende el LED verde; si no, se imprime “ACCESO DENEGADO” y el LED verde se queda apagado. Como indicador adicional se enciende el LED rojo.
- **Temporización con millis():** encenderLED() guarda el instante en tiempoEncendido y activa la bandera ledEncendido. Al inicio de cada loop() se revisa si ya pasaron 2000 ms y, en ese caso, se apagan los LEDs. El programa nunca se detiene a esperar, por lo que sigue leyendo etiquetas mientras un LED está encendido.
- **Supervisión del lector:** cada 5 s, también con millis(), revisarLector() vuelve a leer la versión y la ganancia de la antena. Si el lector deja de responder se muestra un aviso, y si se reinició por una caída de voltaje (la ganancia regresa a su valor por defecto) se vuelve a configurar automáticamente.

El fragmento principal que controla el LED sin usar de lay() es el siguiente:

```
if (ledEncendido && (millis() - tiempoEncendido >= TIEMPO_LED)) {  
    digitalWrite(LED_VERDE, LOW);  
    digitalWrite(LED_ROJO, LOW);  
    ledEncendido = false;  
}  
if (!rfid.PICC_IsNewCardPresent()) return;    // no bloquea  
if (!rfid.PICC_ReadCardSerial()) return;
```

Por ser un UNO R4 WiFi, cuyo puerto serie es USB nativo, se agregó la línea while (!Serial && millis() < 3000); para no perder los primeros mensajes del Monitor Serie.

5. Evidencia del circuito y del monitor serie funcionando

En la Figura 2 se observa el Monitor Serie durante las pruebas: cada lectura muestra el UID en hexadecimal y el resultado de la comparación. El llavero con UID 99 EB 7B 63 obtiene “ACCESO PERMITIDO” y el llavero con UID 16 D7 DC A1 obtiene “ACCESO DENEGADO”.

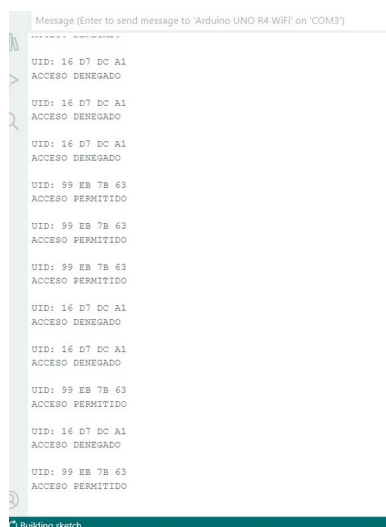


Figura 2. Monitor Serie: UID de cada llavero y resultado del control de acceso.

A continuación se muestra la evidencia fotográfica del circuito armado y de la respuesta de los LEDs con cada llavero.

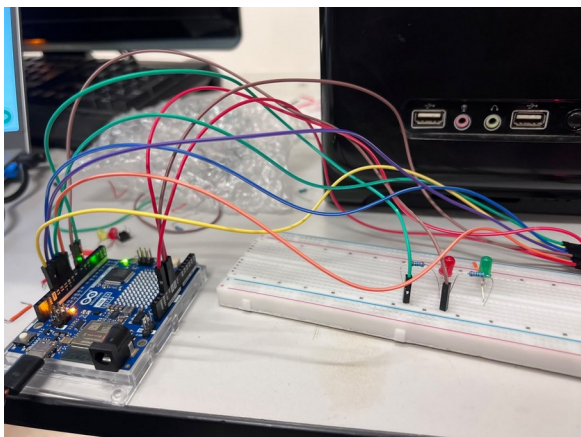


Figura 3. Conexiones SPI y de alimentación en el Arduino UNO R4 WiFi.

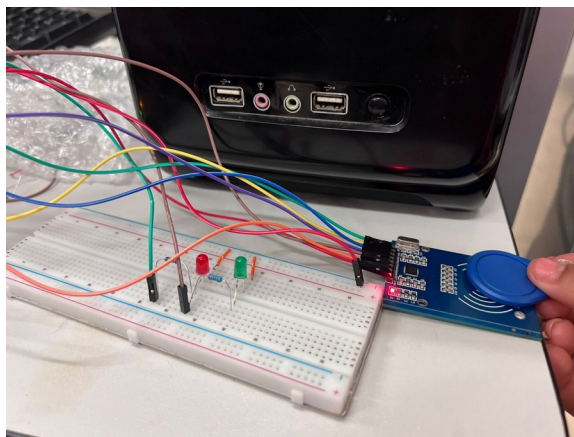


Figura 4. Lector RC522 y LEDs en la protoboard.

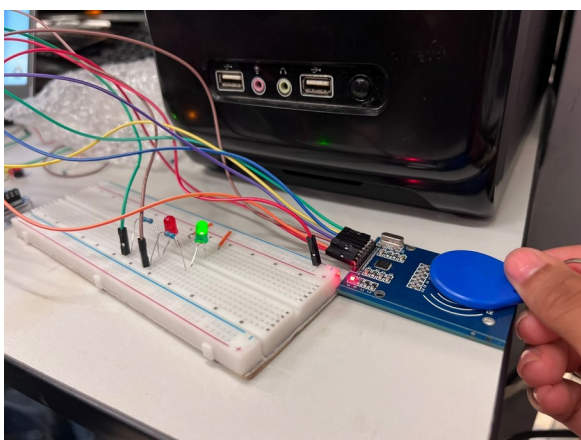


Figura 5. Llavero autorizado: se enciende el LED verde.

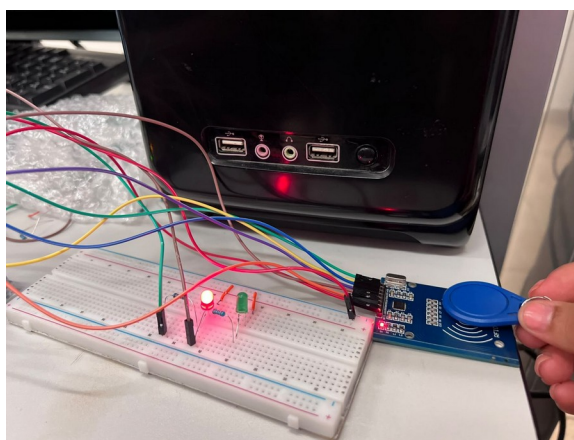


Figura 6. Llavero no autorizado: el LED verde queda apagado y se enciende el rojo.

El video con el funcionamiento de la práctica está disponible en: <https://youtu.be/o6up5xnkzOA>

6. Análisis de resultados

Se leyeron los dos llaveros y se anotó el UID de cada uno. Como ambos son llaveros azules iguales, se identifican por su UID. El primero se eligió como etiqueta autorizada y su UID se guardó dentro del programa.

Tabla II. UID leído de cada etiqueta.

Etiqueta	UID (hexadecimal)	Resultado en el Monitor Serie
Llavero 1 (autorizado)	99 EB 7B 63	ACCESO PERMITIDO
Llavero 2	16 D7 DC A1	ACCESO DENEGADO

Tabla III. Pruebas solicitadas y resultado obtenido.

Prueba	Resultado
Acercar la etiqueta autorizada	Se muestra “ACCESO PERMITIDO”, el LED verde enciende y se apaga solo a los 2 segundos.

Prueba	Resultado
Acercar la etiqueta no autorizada	Se muestra "ACCESO DENEGADO" y el LED verde no enciende; como indicador se enciende el LED rojo.
Con el LED encendido, acercar la etiqueta no autorizada	El programa sigue leyendo durante los 2 s porque no usa <code>delay()</code> : se muestra "ACCESO DENEGADO" y <code>encenderLED()</code> apaga el verde y reinicia el conteo.
Desconectar MISO (D12) y reiniciar	Esta prueba no se realizó físicamente. En la pregunta 4 del cuestionario se explica el comportamiento que produce el programa en ese caso.

Observaciones sobre el comportamiento del sistema:

- Los dos UID tienen 4 bytes, que es el tamaño típico de las etiquetas MIFARE Classic de 1 KB; la función `uidCoincide()` revisa primero el tamaño y después cada byte.
- Las lecturas repetidas del mismo llavero siempre dieron el mismo UID, lo que confirma que el número es fijo y sirve para identificar cada etiqueta.
- El Monitor Serie registró lecturas alternadas entre ambos llaveros sin que el programa se trabara, gracias a que la lectura y el apagado del LED se revisan en cada vuelta de `loop()`.
- Leer el registro de versión al arrancar permite distinguir entre un problema de comunicación (valor 0x00 o 0xFF) y un lector funcionando correctamente.
- Si el lector se reinicia por una caída de voltaje, pierde su configuración (por ejemplo, la ganancia de la antena); para evitar que se quede así, se agregó `revisarLector()`, que lo detecta y lo vuelve a configurar sin reiniciar el Arduino.

7. Cuestionario

1. ¿Qué significan las siglas SCK, MOSI, MISO y CS?

- **SCK (Serial Clock):** reloj serie. Lo genera el maestro (el Arduino) y marca el momento en que se envía y se lee cada bit.
- **MOSI (Master Out, Slave In):** salida del maestro, entrada del esclavo. Por aquí el Arduino le manda al lector los comandos y las direcciones de los registros. En la nomenclatura actual de Arduino se llama COPI (Controller Out, Peripheral In).
- **MISO (Master In, Slave Out):** entrada del maestro, salida del esclavo. Por aquí el lector le devuelve datos al Arduino. También se le llama CIPO (Controller In, Peripheral Out).
- **CS (Chip Select):** selección de chip, también llamado SS (Slave Select). Es activo en bajo: cuando el Arduino lo pone en LOW, ese dispositivo atiende al bus; cuando está en HIGH, lo ignora.

2. ¿Por qué el pin del módulo dice "SDA" si no es I2C? ¿Qué función cumple en realidad?

Porque el chip MFRC522 puede trabajar con tres interfaces (SPI, I2C y UART) y varios de sus pines tienen más de una función. El fabricante del módulo rotuló ese pin con su nombre en modo I2C, donde sería SDA (Serial Data). En el módulo RC522 el chip está configurado por hardware para usar SPI, así que ese pin funciona como **NSS o CS (selección del esclavo)**. Por eso en el código se conecta al pin 10 como `SS_PIN`: cuando el Arduino lo pone en bajo, el lector responde a las transferencias SPI.

3. ¿Por qué línea viaja el UID del lector al Arduino?

Por la línea **MISO**. La etiqueta envía su UID al lector por radiofrecuencia y el MFRC522 lo guarda en su memoria interna (FIFO). Para obtenerlo, el Arduino manda por MOSI la dirección del registro que quiere leer y, con los pulsos de SCK, recibe los bytes del UID por MISO. MOSI solo lleva información del Arduino hacia el lector.

4. ¿Qué pasó al desconectar MISO y por qué?

Sin MISO, el Arduino todavía puede enviar comandos al lector (SCK, MOSI y SS siguen conectados), pero no recibe ninguna respuesta. La entrada D12 queda al aire y cada lectura devuelve 0x00 o 0xFF, según el nivel en que quede la línea. Por eso, al reiniciar el Arduino, la lectura del registro de versión da uno de esos valores y el programa muestra “ERROR: No hay comunicacion con el lector RC522. Revisen las conexiones (SPI, 3.3V, GND).” Tampoco se detecta ninguna etiqueta, porque la respuesta del lector nunca llega, y cada 5 s aparece “AVISO: se perdio la comunicacion con el lector.” El LED del módulo sigue encendido porque la alimentación es correcta: la falla es de comunicación, no de energía. Al volver a conectar MISO y reiniciar, el lector responde de nuevo y la lectura funciona con normalidad.

Nota: esta prueba no se realizó con el circuito físico; el comportamiento descrito corresponde a lo que hace el programa cuando la línea MISO no está conectada.

5. ¿Por qué no se permitió usar delay() en esta práctica? ¿Qué problema habría causado?

Porque delay() es bloqueante: mientras transcurre la espera, el microcontrolador no ejecuta nada más del loop(). Si después de encender el LED se hubiera usado delay(2000), durante esos 2 segundos el programa no habría revisado si había una etiqueta nueva. Al acercar la tarjeta no autorizada con el LED encendido no habría aparecido “ACCESO DENEGADO” en ese momento, sino hasta que terminara la espera, y el sistema parecería trabado; tampoco se habría supervisado el lector.

Con millis() se guarda el instante en que se encendió el LED y en cada vuelta del loop() solo se compara el tiempo transcurrido; el ciclo se repite miles de veces por segundo y el Arduino atiende las lecturas sin interrupciones. Además, la resta millis() - tiempoEncendido con variables unsigned long sigue funcionando cuando millis() se desborda, aproximadamente a los 49.7 días. Los únicos delay() del programa son pausas de 50 ms y 10 ms dentro de iniciarLector(), necesarias para el reinicio del chip, que solo se ejecutan al arrancar o cuando el lector se reinicia.

6. Si quisieran conectar un segundo lector RC522 al mismo Arduino, ¿qué cables compartirían y cuál tendría que ser diferente?

Compartirían las líneas del bus: **SCK (D13), MOSI (D11) y MISO (D12)**, además de 3.3 V y GND. Lo que tendría que ser diferente es la línea **SDA/SS (CS)**: cada lector necesita su propio pin de selección, por ejemplo D10 para el primero y D8 para el segundo, para que el Arduino active solo al lector con el que quiere hablar y el otro deje libre el bus. En el código se crearía un segundo objeto, por ejemplo `MFR522 rfid2(8, RST_PIN_2);`.

El pin RST podría compartirse, pero en este programa iniciarLector() da un pulso de reinicio por RST, y si fuera compartido reiniciaría a los dos lectores a la vez. Por eso conviene usar un pin RST distinto para cada uno, o inicializar ambos al mismo tiempo.

8. Conclusiones individuales

Ximena

Con esta práctica entendí cómo se reparten las tareas las líneas del bus SPI: el Arduino genera el reloj, manda los comandos por MOSI y recibe las respuestas por MISO, mientras que la línea SS decide con qué dispositivo se habla. Me llamó la atención que, a diferencia de I2C, aquí no se usan direcciones, sino un pin de selección para cada dispositivo.

También me quedó claro que el UID es un número fijo de cada etiqueta: aunque los dos llaveros se ven iguales, el lector los distingue sin problema por su UID, y eso es lo que permite usarlos como llave en un control de acceso.

Carlos Tadeo

Esta práctica me ayudó a entender por qué no se debe usar `delay()` cuando el sistema tiene que seguir atento a lo que pasa. Con `millis()` el LED se apaga solo a los 2 segundos y, mientras tanto, el Arduino sigue leyendo etiquetas; en el Monitor Serie se ve cómo las lecturas de los dos llaveros se registran una tras otra sin que el programa se detenga.

Además, me pareció útil leer el registro de versión del lector al arrancar, porque permite saber de inmediato si hay comunicación. Si la línea MISO falla, el Arduino no recibe nada y lee 0x00 o 0xFF, y el programa lo avisa en lugar de quedarse sin responder.

Dulce Princesa

Considero que lo más importante de esta práctica fue cuidar la conexión física del lector. El RC522 debe alimentarse con 3.3 V y no con 5 V, y la línea que en el módulo dice SDA en realidad funciona como selección del lector en SPI. Además, como el lector puede reiniciarse por una caída de voltaje y perder su configuración, agregamos una revisión cada 5 segundos que lo detecta y lo vuelve a configurar sin reiniciar el Arduino.

Adicionalmente, me di cuenta de que este mismo esquema se puede ampliar: para agregar un segundo lector bastaría con compartir SCK, MOSI y MISO y asignarle otro pin de selección. Esto me hizo ver lo práctico que es SPI para conectar varios dispositivos rápidos a un mismo microcontrolador.

Referencias

[1] Arduino, “Arduino UNO R4 WiFi”, documentación oficial. [En línea]. Disponible en: <https://docs.arduino.cc/hardware/uno-r4-wifi/>

[2] Arduino, “SPI”, referencia oficial del lenguaje Arduino. [En línea]. Disponible en: <https://docs.arduino.cc/language-reference/en/functions/communication/SPI/>

[3] NXP Semiconductors, “MFRC522 Standard performance MIFARE and NTAG frontend”, hoja de datos. [En línea]. Disponible en: <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>

[4] GithubCommunity, “MFRC522 – Arduino RFID Library for MFRC522”. [En línea]. Disponible en: <https://github.com/miguelbalboa/rfid>

[5] Arduino, “millis()”, referencia oficial del lenguaje Arduino. [En línea]. Disponible en: <https://docs.arduino.cc/language-reference/en/functions/time/millis/>